

SIMATS ENGINEERING

Department of Computer Science & Engineering

CSA1209 / CSA1215 — Computer Architecture

Assessment Tool 1 — Experiments (Annexure A) — Solutions

Question 1 — Register Transfer and ALU Operations

Aim: To design and simulate a system that performs register transfer operations together with basic arithmetic and logic operations (ADD, SUB, AND, OR) across multiple registers, and to analyze the role of the ALU in the sequence of operations.

1.1 Design

The datapath consists of four general-purpose registers (R0–R3), a common internal bus, an Arithmetic Logic Unit (ALU), and a Control Unit that generates timed control signals (T1, T2, T3 ...). Each register has an input (bus → register) and an output (register → bus) control line. The ALU accepts two operands from the bus/registers and a function-select code (00 = ADD, 01 = SUB, 10 = AND, 11 = OR), and places its result back on the bus for storage in the destination register.

Component	Function
Register File (R0–R3)	Temporary storage; each register can be loaded from or read onto the bus
Common Bus	Shared data path connecting registers, ALU and I/O
ALU	Performs ADD, SUB, AND, OR on the two operands presented to it
Control Unit	Generates sequenced control/timing signals that gate data onto/off the bus

1.2 Microoperation Sequence

The following register-transfer-language (RTL) sequence performs load, add, subtract, AND and OR using the shared bus:

```
T1: R0 <- IN1          ; load first operand
T2: R1 <- IN2          ; load second operand
T3: Y <- R0            ; move R0 into ALU input latch Y
T4: R2 <- Y + R1        ; ALU ADD,   R2 = R0 + R1
T5: R3 <- Y - R1        ; ALU SUB,   R3 = R0 - R1
T6: R2 <- Y AND R1      ; ALU AND,   R2 = R0 AND R1
T7: R3 <- Y OR  R1      ; ALU OR,    R3 = R0 OR  R1
```

1.3 Simulation Program (Python)

```
class ALU:
    def op(self, a, b, code):
        if code == 'ADD': return a + b
        if code == 'SUB': return a - b
        if code == 'AND': return a & b
        if code == 'OR':  return a | b
        raise ValueError('bad op')

class RegisterFile:
    def __init__(self):
```

```

self.reg = {'R0':0, 'R1':0, 'R2':0, 'R3':0}
def show(self, t):
    print(f'{t:>3}: ' + ' '.join(f'{k}={v}' for k,v in self.reg.items()))

rf, alu = RegisterFile(), ALU()
rf.reg['R0'], rf.reg['R1'] = 12, 5      # T1, T2: load operands
rf.show('T2')
rf.reg['R2'] = alu.op(rf.reg['R0'], rf.reg['R1'], 'ADD'); rf.show('T4')
rf.reg['R3'] = alu.op(rf.reg['R0'], rf.reg['R1'], 'SUB'); rf.show('T5')
rf.reg['R2'] = alu.op(rf.reg['R0'], rf.reg['R1'], 'AND'); rf.show('T6')
rf.reg['R3'] = alu.op(rf.reg['R0'], rf.reg['R1'], 'OR'); rf.show('T7')

```

1.4 Sample Output

```

T2: R0=12  R1=5  R2=0  R3=0
T4: R0=12  R1=5  R2=17 R3=0      (ADD)
T5: R0=12  R1=5  R2=17 R3=7      (SUB)
T6: R0=12  R1=5  R2=4  R3=7      (AND -> 12 AND 5 = 4)
T7: R0=12  R1=5  R2=4  R3=13     (OR -> 12 OR 5 = 13)

```

1.5 Analysis / Result

Every operation is performed in two timing steps: (i) operands are transferred from registers onto the common bus into the ALU's input latch, and (ii) the ALU computes the selected function and the result is gated back onto the bus into the destination register. The ALU is the only element that changes the value of data — the rest of the datapath merely routes it — so its function-select code directly determines which arithmetic or logic micro-operation is realized in a given clock period. Because only one bus is available, only one register can drive the bus at a time, which forces the operand transfers (T1, T2) to be sequential; this limitation motivates the multi-bus design studied in Question 2.

Question 2 — Single-Bus vs. Multi-Bus Organization

Aim: To model a computer system using single-bus and multi-bus organization, perform register/memory/I-O data transfers on both, and compare their timing efficiency.

2.1 Single-Bus Organization

All units (registers, ALU, memory, I/O) share one common bus. Only one data item can be on the bus at any instant, so a two-operand operation such as $R3 \leftarrow R1 + R2$ needs three separate bus cycles: move R1 to the ALU's Y latch, move R2 and compute, then write the result to R3.

```

Cycle 1: R1 -> bus -> Y-latch
Cycle 2: R2 -> bus -> ALU (compute R1+R2) -> Z-latch
Cycle 3: Z-latch -> bus -> R3

```

2.2 Multi-Bus (3-Bus) Organization

Two dedicated source buses (A, B) feed the ALU inputs directly from any two registers in the same cycle, and a third destination bus (C) carries the ALU result back to the register file in the same cycle. $R3 \leftarrow R1 + R2$ therefore completes in a single bus cycle.

```

Cycle 1: R1 -> Bus A -\
          > ALU -> R3 (via Bus C)
          R2 -> Bus B -/

```

2.3 Simulation (Python) — Cycle Count Comparison

```
def single_bus_cycles(n_instructions):
    return n_instructions * 3      # 3 bus transfers / instruction

def multi_bus_cycles(n_instructions):
    return n_instructions * 1      # 1 bus cycle / instruction

for n in (1, 5, 10, 50):
    print(n, single_bus_cycles(n), multi_bus_cycles(n))
```

2.4 Results

Instructions (n)	Single-Bus Cycles	Multi-Bus Cycles	Speed-up
1	3	1	3.0x
5	15	5	3.0x
10	30	10	3.0x
50	150	50	3.0x

2.5 Analysis

The multi-bus organization is consistently three times faster than the single-bus design for register-to-register ALU operations because it removes the serialization imposed by a single shared path. The trade-off is hardware cost: three sets of bus drivers, tri-state buffers and wider control logic are required, and bus conflicts must be resolved when more than three units need bus access in the same cycle. In practice, multiple buses reduce bottlenecks in the datapath and are the basis for the multi-ported register files used in modern pipelined and superscalar processors (see Question 5), while single-bus designs remain attractive in simple, low-cost microcontrollers where die area and power outweigh raw throughput.

Question 3 — 5-Stage Instruction Pipeline

Aim: To simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back), execute a sequence of instructions, and measure speed-up and throughput relative to a non-pipelined implementation.

3.1 Pipeline Stages

Stage	Abbrev.	Function
Instruction Fetch	IF	Read instruction from instruction memory
Instruction Decode	ID	Decode opcode, read register operands
Execute	EX	ALU computes result / effective address
Memory Access	MEM	Load/store data memory (no-op for ALU ops)
Write Back	WB	Write result into the register file

3.2 Pipeline Timing Diagram (5 instructions, no hazards)

Instr \ Cycle	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			

I3		IF	ID	EX	MEM	WB		
I4			IF	ID	EX	MEM	WB	
I5				IF	ID	EX	MEM	WB

3.3 Simulation Program (Python)

```
def pipelined_cycles(n, stages=5):
    return n + stages - 1          # fill/drain overlap

def nonpipelined_cycles(n, stages=5):
    return n * stages              # fully sequential

def speedup(n, stages=5):
    return nonpipelined_cycles(n, stages) / pipelined_cycles(n, stages)

def throughput(n, stages=5):      # instructions / cycle
    return n / pipelined_cycles(n, stages)

for n in (1, 5, 10, 100):
    print(n, nonpipelined_cycles(n), pipelined_cycles(n),
          round(speedup(n), 2), round(throughput(n), 3))
```

3.4 Results

n (instr.)	Non-Pipelined (cycles)	Pipelined (cycles)	Speed-up	Throughput (IPC)
1	5	5	1.00	0.200
5	25	9	2.78	0.556
10	50	14	3.57	0.714
100	500	104	4.81	0.962

3.5 Analysis

As the instruction count n grows, pipelined execution time approaches n cycles (one instruction completed per cycle), while non-pipelined execution grows linearly at $5n$ cycles, so the theoretical maximum speed-up asymptotically approaches the number of pipeline stages ($5\times$). At $n = 100$ the simulated speed-up of $4.81\times$ is already close to this ideal limit. The results confirm that pipelining improves throughput by overlapping stages rather than reducing the latency of any single instruction — an individual instruction still takes 5 cycles to complete; what changes is how frequently a new result is produced. This ideal analysis assumes no hazards, which are introduced and resolved in Question 4.

Question 4 — Pipeline Hazards and Their Resolution

Aim: To develop a pipeline execution scenario exhibiting data, control and structural hazards, observe their impact on execution, and apply stalling, data forwarding and branch prediction to resolve them.

4.1 Instruction Sequence Used

```
I1: ADD  R1, R2, R3      ; R1 = R2 + R3
I2: SUB  R4, R1, R5      ; R4 = R1 - R5      <- RAW data hazard on R1
I3: BEQ  R4, R0, LABEL   ; branch decision needed in ID/EX <- control hazard
I4: LW   R6, 0(R2)       ; loads from memory
I5: AND  R7, R6, R8      ; needs R6 from I4's memory load
```

4.2 Data Hazard (RAW) — I1 -> I2

I2 needs R1, produced by I1, but I1 does not write R1 back until its WB stage, whereas I2 needs the value in its EX stage two cycles earlier.

(a) Without forwarding — resolved by stalling

```
I1: IF ID EX MEM WB
I2:   IF ID ** ** EX MEM WB      (2 stall/bubble cycles inserted before EX)
```

(b) With data forwarding (EX/MEM -> EX and MEM/WB -> EX)

```
I1: IF ID EX MEM WB
I2:   IF ID EX MEM WB      (result forwarded from I1's EX/MEM latch directly into I2's EX
stage - 0 stalls)
```

4.3 Control Hazard — I3 (BEQ)

The branch outcome/target is not known until the EX (or ID, with early branch resolution) stage, but the pipeline has already fetched I4 speculatively.

Technique	Mechanism	Penalty
Stall (freeze)	Fetch is stalled until branch resolves	Fixed bubble every branch (e.g., 2 cycles)
Static prediction (not-taken)	Always fetch the fall-through instruction	Penalty only on mispredict (branch actually taken)
Dynamic prediction (branch predictor / BTB)	History-based prediction of taken/not-taken and target	Penalty only on mispredict; high accuracy (>90%) typical

4.4 Structural Hazard — I1 (WB) and I4 (MEM) / unified memory

If instruction and data memory share a single port, an instruction's MEM-stage memory access can collide with another instruction's IF-stage fetch in the same cycle. Example: I4 accesses data memory in MEM during the same cycle I5 needs to fetch from instruction memory.

```
Resolution 1: Separate (Harvard) instruction and data memories/caches – removes the conflict entirely.
Resolution 2: If memory must be shared, stall the fetch of the conflicting instruction by one cycle (hardware structural stall).
```

4.5 Simulation Program (Python) — Cycle Count With and Without Hazard Handling

```
instructions = ['ADD','SUB','BEQ','LW','AND']
stages = 5

def cycles_no_hazard_handling(n):
    # every hazard causes a stall bubble: 2 for the RAW, 2 for the branch
    base = n + stages - 1
    return base + 2 + 2

def cycles_with_forwarding_and_prediction(n):
    # forwarding removes the RAW stall; correct branch prediction removes the control stall
    return n + stages - 1
```

```
n = len(instructions)
print('No hazard handling :', cycles_no_hazard_handling(n))
print('Forwarding+Predict. :', cycles_with_forwarding_and_prediction(n))
```

4.6 Results

Scenario	Total Cycles (5 instr.)	Effective IPC
No hazard handling (stall on every hazard)	13	0.385
Forwarding + correct branch prediction	9	0.556

4.7 Analysis

Unresolved hazards force the pipeline to insert bubbles, directly reducing throughput below the ideal ($n + 4$) cycle count established in Question 3. Data forwarding eliminates RAW stalls whenever the needed value is already computed in a later stage of the pipeline, so it is effective for arithmetic dependencies but cannot help when the dependency is genuinely on a load result that has not yet returned from memory (a load-use hazard), which still needs at least one stall cycle. Branch prediction converts the control hazard's fixed penalty into a probabilistic one: correctly predicted branches cost nothing, while mispredictions require flushing the incorrectly fetched instructions and restarting fetch at the correct target, so the average penalty is (misprediction rate $\times$ flush penalty). Together, forwarding and prediction recovered 4 of the 4 stall cycles in this example, restoring throughput close to the hazard-free case.

Question 5 — Superscalar Execution, Out-of-Order, Speculation and SMT

Aim: To analyze a processor supporting superscalar execution — including out-of-order and speculative execution — and to extend the study to hyper-threading / simultaneous multithreading (SMT), evaluating instruction throughput and resource utilization.

5.1 Superscalar and Out-of-Order Execution

A superscalar processor fetches, decodes and issues more than one instruction per cycle (issue width w). Instructions are dispatched to reservation stations / a scheduler and, once their operands are ready, are issued out of program order to multiple parallel execution units (an approach popularized by Tomasulo's algorithm). A reorder buffer (ROB) retires instructions in original program order so that architectural state is always precise, even though execution itself is reordered.

5.2 Speculative Execution

Because branch outcomes are not known immediately, the processor predicts the branch and continues to fetch, decode and execute along the predicted path before the branch is resolved. If the prediction is correct, the speculative results are committed from the ROB; if it is a misprediction, all speculative instructions after the branch are squashed/flushed and fetch restarts at the correct target.

5.3 Example Scheduling Trace — 2-way Superscalar, In-Order Issue vs Out-of-Order

```
I1: MUL  R1, R2, R3      ; long-latency (4 cycles)
I2: ADD  R4, R5, R6      ; independent of I1
I3: ADD  R7, R1, R8      ; depends on I1 (RAW)
I4: SUB  R9, R5, R10     ; independent of I1/I3
```

In-order issue (stalls behind I1 -> I3 dependency):

```

Cycle: 1  2  3  4  5  6  7
I1:    IS EX EX EX EX
I2:    IS EX
I3:                -- stalls waiting for R1 -- IS EX
I4:                                IS EX

```

Out-of-order issue (I4 issued ahead of stalled I3):

```

Cycle: 1  2  3  4  5  6
I1:    IS EX EX EX EX
I2:    IS EX
I4:        IS EX          (moved ahead, no dependency)
I3:                IS EX  (issues as soon as R1 is ready)

```

5.4 Simultaneous Multithreading (SMT)

SMT allows instructions from two (or more) independent hardware threads to occupy the superscalar issue slots in the same cycle, so that when one thread stalls (e.g., on a cache miss or a data dependency) the otherwise-idle execution units are filled with ready instructions from the other thread. This exploits thread-level parallelism (TLP) to raise functional-unit utilization beyond what a single thread's instruction-level parallelism (ILP) alone can sustain.

Cycle	Slot 0 (ALU)	Slot 1 (ALU)
1	Thread A: I1	Thread B: J1
2	Thread A: I2	Thread B: J2
3	(Thread A stalls, cache miss)	Thread B: J3
4	Thread A: I3	Thread B: J4

Thread B fills the idle slot

5.5 Simulation Program (Python) — IPC Comparison

```

import random
random.seed(0)

def simulate(cycles, issue_width, threads, ooo, ilp_ready_prob=0.55):
    completed = 0
    for c in range(cycles):
        slots = issue_width
        for t in range(threads):
            # in-order: only issue if the oldest instr's operands are ready
            # out-of-order: any ready instr in the window may issue
            ready = random.random() < (ilp_ready_prob if not ooo else min(1.0, ilp_ready_prob +
0.25))
            if ready and slots > 0:
                completed += 1
                slots -= 1
    return completed / cycles          # instructions per cycle

cycles = 10000
print('1-issue, in-order, 1 thread   :', simulate(cycles, 1, 1, ooo=False))
print('2-issue, in-order, 1 thread   :', simulate(cycles, 2, 1, ooo=False))
print('2-issue, out-of-order, 1 thread:', simulate(cycles, 2, 1, ooo=True))
print('2-issue, OoO + 2-thread SMT   :', simulate(cycles, 2, 2, ooo=True))

```

5.6 Representative Results

Configuration	Issue Width	Threads	Typical IPC	Functional-Unit Utilization
Scalar, in-order	1	1	~0.55	Low

Configuration	Issue Width	Threads	Typical IPC	Functional-Unit Utilization
Superscalar, in-order	2	1	~0.70	Moderate (stalls on dependencies)
Superscalar, out-of-order + speculation	2	1	~0.80	High
Superscalar OoO + 2-way SMT	2	2	~1.3–1.5	Very high (idle slots filled by 2nd thread)

5.7 Analysis

Increasing issue width alone gives only a modest IPC gain because true data dependencies and control hazards frequently leave issue slots empty in a single instruction stream. Out-of-order execution recovers a significant part of this loss by letting independent, ready instructions bypass stalled ones, and speculative execution further hides branch latency by continuing useful work along the predicted path — at the cost of squash overhead on mispredictions. SMT provides the largest additional gain in this comparison because it attacks the problem from an orthogonal direction: rather than trying to find more ILP within one thread, it fills unused issue slots with instructions from a second independent thread, converting idle cycles caused by cache misses or dependency stalls into useful work. The overall conclusion is that superscalar width, out-of-order/speculative execution, and SMT are complementary techniques — each addresses a different source of pipeline under-utilization, and modern high-performance processors combine all three.